Tema 2 - Interfaces de usuario.

Layouts, controles y menús

Índice

1. INTRODUCCIÓN 3

1.1. PARADIGMA DECLARATIVO VS IMPERATIVO TRADICIONAL 3

1.2. CICLO DE RECOMPOSICIÓN Y GESTIÓN DEL ESTADO 4

1.2.1. Conservación del estado 5

1.2.2. Vinculación de estado 6

2. FUNDAMENTOS DE LA INTERFAZ @COMPOSABLE 6

2.1. FUNCIONES @COMPOSABLE 6

2.2. MODIFIERS 7

3. DISPOSICIÓN Y CONTENEDORES 8

3.1. COLUMN 9

3.2. ROW (DISPOSICIÓN HORIZONTAL) 10

3.3. BOX 11

3.4. SPACER 12

3.5. ESTRUCTURAS AVANZADAS DE DISEÑO 12

3.6. SCAFFOLD Y SURFACE 13

4. COMPONENTES Y CONTROLES INTERACTIVOS 13

4.1. TEXT 13

4.2. BOTONES 14

4.2.1. Button 14

4.2.2. IconButton e Icon 15

4.2.3. FloatingActionButton (FAB) 15

4.3. CAMPOS DE ENTRADA 15

4.3.1. KeyboardOptions 16

4.4. SELECCIÓN Y OPCIONES 16

4.4.1. Checkbox 16

4.4.2. RadioButton 17

4.4.3. Switch 17

4.4.4. Menús desplegables de selección 18

5. COLECCIONES DINÁMICAS Y LISTAS PEREZOSAS 18

5.1. INTRODUCCIÓN AL RENDERIZADO PEREZOSO 19
5.2. LISTAS VERTICALES 19
5.3. CUADRÍCULAS 21
5.4. OPTIMIZACIÓN CON CLAVES ÚNICAS 22

6. MENÚS Y BARRAS DE ACCIONES 23

6.1. BARRA SUPERIOR 23
6.2. MENÚS DESPLEGABLES 24
6.3. MENÚS CONTEXTUALES 26

1. Introducción

La **interfaz de usuario** o *UI* constituye el punto de interacción directo entre el usuario y la aplicación móvil. A lo largo de la evolución de Android, la construcción de interfaces ha experimentado una progresiva transformación arquitectónica: desde el modelo clásico basado en jerarquías de objetos mutables definidos en archivos XML hasta el estándar contemporáneo basado en el paradigma declarativo y reactivo propio de ***Jetpack Compose***.

1.1. Paradigma declarativo vs Imperativo tradicional

Para comprender el desarrollo de interfaces actual en la plataforma Android, es necesario contrastar los dos modelos de construcción visual que coexisten en la misma. Históricamente, Android separaba la interfaz y la lógica de programación en dos capas diferenciadas:

- Por un lado, se realizaba la **definición de vistas en XML,** declarando árboles jerárquicos de etiquetas que heredaban de las clases `View` (botones, textos) y `ViewGroup` (contenedores como LinearLayout o ConstraintLayout).
- Por el otro, se llevaba a cabo una **manipulación manual desde el código.** Al instanciar la actividad, el archivo XML se *inflaba* en memoria mediante el método `setContentView()`. El desarrollador buscaba las referencias a dichos objetos (con métodos como `findViewById()`) y los manipulaba de forma imperativa en respuesta a eventos.

```
val textView = findViewById<TextView>(R.id.tvMensaje)
val boton = findViewById<Button>(R.id.btnActualizar)
boton.setOnClickListener {
    textView.text = "Texto actualizado manualmente"
    textView.visibility = View.VISIBLE
}
```

Al coexistir dos concepciones diferentes de las diferentes vistas (desde el ámbito de XML y el ámbito del código), el programador debía sincronizar manualmente la vista cada vez que los datos cambiaban. Esto podía provocar multitud de **errores**, tales como inconsistencias visuales, fugas de memoria o excepciones por `NullPointerException`.

En el **modelo declarativo** de Jetpack Compose, sin embargo, la interfaz se concibe como una función matemática del estado actual de la aplicación. En lugar de decirle al sistema la secuencia de pasos para modificar un componente existente, el desarrollador describe cómo debe representarse la interfaz para un determinado valor de datos:

```
@Composable
fun SaludoDinamico(nombre: String) {
    Text(text = "Hola, $nombre")
}
```

Este enfoque facilita el desarrollo de la interfaz, eliminando la necesidad de circular de los archivos XML al código con la lógica de programación. Además, el paradigma declarativo es altamente modularizable y reutilizable.

1.2. Ciclo de recomposición y gestión del estado

En Jetpack Compose, el **estado** representa cualquier dato susceptible de cambiar a lo largo del ciclo de vida de la aplicación: el texto introducido en un formulario, la pulsación de una casilla una lista de productos devuelta por un servidor, etc.

La **recomposición** es el mecanismo inteligente mediante el cual Compose vuelve a invocar las funciones `@Composable` cuyos datos de entrada han cambiado, actualizando únicamente las partes visuales afectadas en pantalla y descartando las que no han sufrido modificaciones. De esta manera, si una columna contiene diez elementos y solo cambia el dato del último, únicamente se recompondrá dicho elemento, evitando redibujar la jerarquía completa.

Las funciones `@Composable` pueden ejecutarse con mucha frecuencia, en cualquier orden o incluso en hilos de ejecución paralelos. Por ello, deben ser funciones sencillas y que no incurran en efectos secundarios no controlados.

1.2.1. Conservación del estado

Para que un cambio en una variable desencadene una recomposición, dicha variable debe ser observable por el motor de Compose. Para ello, Android cuenta con dos mecanismos principales:

- `mutableStateOf(valorInicial)`. Envuelve el dato en una estructura observable de lectura/escritura. Cuando se modifica su propiedad *value*, Compose detecta el cambio y programa la recomposición de los componentes que lo estén leyendo.
- `remember{...}`. Permite que un valor sobreviva a través de sucesivas recomposiciones. Sin remember, cada vez que la función composable se volviera a ejecutar, la variable se reinicializaría a su valor predeterminado, perdiendo cualquier interacción del usuario.

```
@Composable
fun ContadorBasico() {
    var contador by remember { mutableStateOf(0) }
    Column(
        modifier = Modifier.padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "Pulsaciones actuales: $contador")
        Spacer(modifier = Modifier.height(8.dp))
        Button(onClick = { contador++ }) {
            Text(text = "Incrementar")
        }
    }
}
```

Aunque `remember` retiene el valor durante las recomposiciones normales, no sobrevive a la destrucción de la actividad por cambios de configuración hardware (como la rotación de pantalla o el cambio de idioma). Para solventar esto, se utiliza `rememberSaveable`, el cual almacena automáticamente el estado en el Bundle `savedInstanceState` de la actividad.

1.2.2. Vinculación de estado

En Compose, la mayoría de los controles no retienen su propio valor. Para que un componente sea interactivo, se aplican dos elementos complementarios: el valor a mostrar, proporcionado por una variable observable, y un evento de actualización. Por ejemplo:

```
@Composable
fun CampoTextoSimple() {
    var texto by remember { mutableStateOf("") } //Variable con el valor
    TextField(
        value = texto,
        onValueChange = { nuevoTexto -> texto = nuevoTexto }, //Evento
        label = { Text("Escribe aquí") }
    )
}
```

2. Fundamentos de la interfaz @Composable

Como ya hemos visto, toda la interfaz de una aplicación moderna de Androis se construye a partir de funciones `@Composable`. A continuación, se analizarán y detallarán los elementos fundamentales de este tipo de funciones.

2.1. Funciones @Composable

Una función `@Composable` es una función estándar de Kotlin marcada con la correspondiente anotación especifica. Su propósito fundamental es describir y emitir nodos de interfaz gráfica a partir de sus parámetros de entrada.

```
@Composable
fun MensajeBienvenida(usuario: String, modifier: Modifier = Modifier) {
    Text(
        text = "¡Hola, $usuario!",
        modifier = modifier
    )
}
```

Este tipo de funciones solo puede ser invocada desde otra función `@Composable` o desde el bloque raíz `setContent{}` de una actividad. No devuelven ninguna referencia de objeto ni puntero visual, ya que su ejecución produce directamente la descripción de los elementos en el árbol de composición.

Es una buena práctica que toda función `@Composable` propia reciba un parámetro `modifier: Modifier = Modifier` con un valor por defecto. Esto permite que el componente padre pueda ajustar sus parámetros sin alterar la lógica interna del componente.

2.2. Modifiers

El objeto `Modifier` es una de las herramientas más importantes de Jetpack Compose. Funciona como una colección ordenada de modificadores que indican a un elemento `@Composable` cómo debe comportarse, posicionarse o dibujarse en pantalla.

Se aplican mediante una sucesión de propiedades encadenadas. Algunas de las más importantes son:

- Dimensiones y tamaño:
  - `.fillMaxSize()`. Ocupa todo el alto y ancho disponible dentro del contenedor padre.
  - `.fillMaxWidth()` / `.fillMaxHeight()`. Ocupa el 100% del ancho o del alto, respectivamente.
  - `.size(ancho, alto)` o `.size([cantidad].dp)`. Asigna dimensiones fijas en unidades dp (Density-independent Pixels).
  - `.width()` / `.height()`. Establece una dimensión fija solo en el eje horizontal o vertical.
- Espaciado interno y separación (padding). En Compose no existe el concepto tradicional de *margin* (margen exterior). Todo el espaciado se gestiona mediante .padding(). Puede aplicarse de forma uniforme (`.padding([cantidad].dp)`), por ejes (`.padding(horizontal = [cantidad].dp, vertical = [cantidad].dp)`) o por lados concretos (`.padding(top = [cantidad].dp)`).
- Alineación y posicionamiento. Con `.align()`se permite a un elemento hijo dentro de un contenedor (como Box, Row o Column) posicionarse en una ubicación concreta (por ejemplo, `Alignment.Center` o `Alignment.End`).

- Apariencia visual:
  - `.background(Color.[color], shape = [forma]([cantidad].dp))`. Define un color o degradado de fondo, opcionalmente recortado a una forma.
  - `.border(width = [cantidad].dp, color = Color.[color])`. Dibuja un borde exterior alrededor del elemento.
  - `.clip([forma]([cantidad].dp))`. Recorta el contenido del componente a la geometría indicada. Resulta útil para redondear imágenes.
- Interacción y eventos. Con `.clickable{}` se convierte cualquier `@Composable` en un elemento pulsable que detecta toques y reproduce el efecto visual táctil.

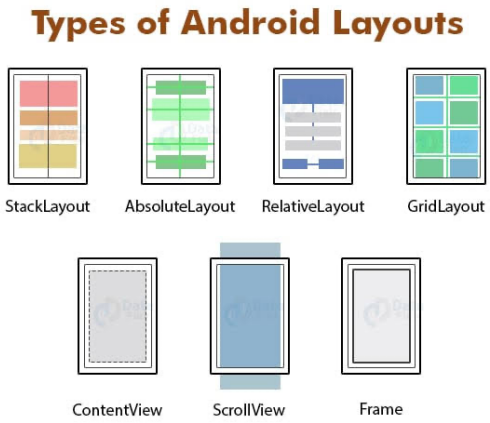
A diferencia de los atributos XML en los que el orden en que se escribían en el archivo era irrelevante, en Compose los modificadores se evalúan secuencialmente de arriba a abajo. **Alterar el orden cambia completamente el resultado visual final**.

```
// Caso A: El fondo cubre el padding (el padding actúa como espacio interior)
Box(
    modifier = Modifier
        .background(Color.Yellow)
        .padding(16.dp)
)

// Caso B: El padding se aplica primero y el fondo después (el padding actúa
como margen exterior)
Box(
    modifier = Modifier
        .padding(16.dp)
        .background(Color.Yellow)
)
```

3. Disposición y Contenedores

En el desarrollo clásico con XML, la pantalla se estructuraba a partir de etiquetas de la clase `ViewGroup` (LinearLayout, FrameLayout, RelativeLayout, etc.). En Jetpack Compose, los contenedores son funciones `@Composable` estándar que definen cómo se posicionan, alinean y distribuyen sus elementos asociados en el espacio de la pantalla.



A continuación, se analizarán todos los elementos de Jetpack Compose que equivalen a las antiguas estructuras de Layout.

3.1. Column

El elemento `Column` organiza los componentes internos en en una única línea vertical donde los elementos se suceden uno tras otro, de arriba hacia abajo. Su equivalencia con el modelo clásico de XML es *LinearLayout* con orientación vertical.

```
@Composable
fun EjemploColumn() {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.SpaceBetween,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Elemento Superior")
        Text("Elemento Central")
        Text("Elemento Inferior")
    }
}
```

Los elementos principales de column son los parámetros *verticalArrangement* y *horizontalAlignment*, que controlan como se reparten los elementos internos a lo largo de la altura y el ancho de la columna, respectivamente.

En *verticalArrangement* existen las opciones `Top`, `Center`, `Bottom`, `SpaceBetween` (equidistantes con el primer elemento arriba del todo y el último abajo del todo), `SpaceEvenly` (repartidos con el mismo espacio entre unos y otros) y `spacedBy([cantidad].dp)` (separación fija).

En *horizontalAlignment* están las opciones `Start`, `CenterHorizontally` y `End` para izquierda, centro y derecha respectivamente.

3.2. Row (disposición horizontal)

Row organiza los componentes internos en una única línea horizontal de izquierda a derecha. Es equivalente a *LinearLayout* con orientación horizontal.

```
@Composable
fun EjemploRow() {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(8.dp),
        horizontalArrangement = Arrangement.SpaceAround,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Button(onClick = { /* Acción 1 / }) { Text("Aceptar") }
        Button(onClick = { / Acción 2 */ }) { Text("Cancelar") }
    }
}
```

De forma similar a `Column`, `Row` admite las propiedades *horizontalArrangement* para distribuir los elementos a lo largo del ancho (`Top`, `Center`, `Bottom`, `SpaceBetween`...) y *verticalAligment* para distribuirlos dentro de la altura de la fila (`Start`, `CenterHorizontally` y `End`).

Cabe destacar también que tanto Row como Column admiten un parámetro específico de Modifier que es el peso de alguno de los componentes internos, que permite destacar uno de éstos por encima de los demás con un modificador numérico:

```
Row(modifier = Modifier.fillMaxWidth()) {
    Text(
        text = "Ocupa el 70%",
        modifier = Modifier.weight(0.7f)
    )
    Text(
        text = "Ocupa el 30%",
        modifier = Modifier.weight(0.3f)
    )
}
```

3.3. Box

El elemento `Box` es equivalente al *FrameLayout*. Permite superponer elementos unos encima de otros en capas en el eje Z, con el último elemento declarado en el bloque visualizándose por encima de los anteriores.

Además de la superposición, `Box` se utiliza habitualmente para posicionar un único componente hijo en una ubicación exacta de la pantalla mediante `contentAlignment`:

```
@Composable
fun EjemploBox() {
    Box( // Alinea el contenido por defecto al centro total
        modifier = Modifier
            .fillMaxSize()
            .background(Color.White),
        contentAlignment = Alignment.Center
    ) {
        Box( // Capa inferior: Imagen o fondo
            modifier = Modifier
                .size(200.dp)
                .background(Color.LightGray)
        )
        Text( // Capa superior 1: Texto centrado
            text = "Capa encima",
            modifier = Modifier.align(Alignment.Center)
        )


        Text( // Capa superior 2: Elemento anclado a la esquina
            text = "Nuevo",
            color = Color.Red,
            modifier = Modifier
                .align(Alignment.TopEnd)
                .padding(8.dp)
        )
    }
}
```

Dentro de un `Box`, cada componente interno puede anular la alineación global mediante el modificador `.align()`, ubicándose en cualquiera de los 9 puntos cardinales (T*opStart, TopCenter, TopEnd, CenterStart, Center, CenterEnd, BottomStart, BottomCenter, BottomEnd*).

3.4. Spacer

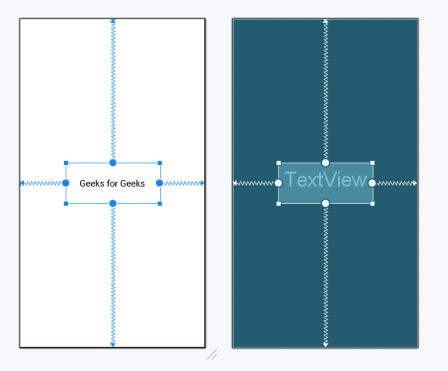
`Spacer` es un composable transparente diseñado específicamente para generar un espacio vacío entre componentes. En lugar de sobrecargar elementos individuales con múltiples llamadas a `.padding()`, el uso de `Spacer` produce un código más legible y desacoplado.

En `Column` se parametriza a través de la llamada `.height([cantidad].dp)` de su *Modifier*, mientras que en `Row` es con `.width([cantidad].dp)`.

```
Column {
    Text("Título principal")
    // Espacio vertical de 16 dp entre el título y la descripción
    Spacer(modifier = Modifier.height(16.dp))
    Text("Descripción secundaria")
}
```

3.5. Estructuras avanzadas de diseño

Aunque la combinación de `Column`, `Row` y `Box` cubre la inmensa mayoría de necesidades de diseño, en casos de interfaces complejas o con requerimientos de alineación cruzada y adaptativa, Compose ofrece herramientas avanzadas como *ConstraintLayout;* está disponible como librería opcional dentro de Compose, y permite anclar elementos unos a otros mediante restricciones relativas, barreras, cadenas... De esta forma, se puede evitar el anidamiento excesivo de filas y columnas en interfaces complejas.



3.6. Scaffold y Surface

El Material Design de Google define una serie de componentes fundamentales para dar soporte a la estructura de una aplicación Android.

`Surface` es un contenedor básico que aplica elevación, color de fondo contextual según el tema y recorte de esquinas. Además, establece automáticamente el color de contraste predeterminado para cualquier texto que se dibuje sobre él.

Por otra parte, `Scaffold` implementa la estructura visual básica de una pantalla de Material Design, ofreciendo espacios predefinidos para integrar múltiples barras, tales como:

- Barra superior de aplicaciones (*topBar*).
- Barra inferior de navegación (*bottomBar*).
- Botón de acción flotante (*floatingActionButton*).
- Ventana de mensajes o avisos temporales (*snackbarHost*).

En las versiones actuales de Android, la instrucción `enableEdgeToEdge()` hace que la ventana de la aplicación se extienda de extremo a extremo, dibujándose por detrás de las barras de estado y de navegación del SO.

El parámetro `innerPadding` propio de `Scaffold` contiene las dimensiones exactas en dp de estas áreas reservadas del sistema. Es **obligatorio** aplicarlo a la raíz del contenido (`Modifier.padding(innerPadding)`) para garantizar que el texto o los botones no queden tapados ni solapados bajo las barras del sistema.

4. Componentes y Controles interactivos

4.1. Text

El elemento `Text` es el componente fundamental para representar **cadenas de texto** en pantalla.

```
Text(
    text = "Texto básico en pantalla",
    color = MaterialTheme.colorScheme.primary,
    fontSize = 18.sp,
    fontWeight = FontWeight.Bold,
    textAlign = TextAlign.Center,
    modifier = Modifier.fillMaxWidth()
)
```

El tamaño de las fuentes se expresa en *sp* (*scale-independent pixels*), de forma que tome como referencia el tamaño de la fuente del sistema.

4.2. Botones

Compose cuenta con una amplia variedad de elementos pulsables según su nivel de relevancia visual y en la jerarquía de la interfaz.

4.2.1. Button

Hay tres tipos de **Button básico** con diferentes características:

- `Button`. Botón con fondo sólido para la acción primaria o más relevante de la pantalla.
- `OutlinedButton`. Botón con borde exterior y fondo transparente, ideal para acciones secundarias.
- `TextButton`. Botón plano sin borde usado frecuentemente en cuadros de diálogo o acciones de bajo énfasis.

```
Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
    Button(onClick = { /* Acción primaria */ }) {
        Text("Guardar")
    }
    OutlinedButton(onClick = { /* Acción secundaria */ }) {
        Text("Revisar")
    }
    TextButton(onClick = { /* Descartar */ }) {
        Text("Cancelar")
    }
}
```

4.2.2. IconButton e Icon

Para botones compactos representados exclusivamente por un **elemento multimedia** de tipo imagen se utiliza `IconButton` conteniendo un `@Composable Icon`.

```
IconButton(onClick = { /* Acción de búsqueda */ }) {
    Icon(
        imageVector = Icons.Default.Search,
        contentDescription = "Buscar usuario"
    )
}
```

4.2.3. FloatingActionButton (FAB)

El **botón de acción flotante** destaca la acción principal de la pantalla por encima del resto del contenido (frecuentemente anclado en la ranura floatingActionButton de un Scaffold).

```
FloatingActionButton(
    onClick = { /* Crear nuevo elemento */ }
) {
    Icon(Icons.Default.Add, contentDescription = "Añadir registro")
}
```

4.3. Campos de entrada

Las entradas de texto se gestionan mediante `TextField` (estilo con fondo relleno) y `OutlinedTextField` (estilo con borde contorneado).

De acuerdo con lo visto en el apartado 1.2.2, estos componentes no almacenan su propio texto, sino que deberían se vinculan a una variable de estado mediante *value* y *onValueChange*:

```
var correo by rememberSaveable { mutableStateOf("") }

OutlinedTextField(
    value = correo,
    onValueChange = { nuevoValor -> correo = nuevoValor },
    label = { Text("Correo electrónico") },
    placeholder = { Text("ejemplo@dominio.com") },
    singleLine = true,
    modifier = Modifier.fillMaxWidth()
)
```

4.3.1. KeyboardOptions

El parámetro *keyboardOptions* instruye al sistema operativo sobre qué tipo de teclado virtual desplegar y qué acción mostrar en la tecla de intro.

- `PersonName`. Automatiza las mayúsculas en la primera letra.
- `Email`. Asegura que el teclado muestra la @ y desactiva las mayúsculas automáticas.
- `Password`[*Visibilidad*]. Formato para contraseñas, con la opción *Visible* para que el usuario la vea mientras teclea y *Masked* para mantenerla oculta.
- `Uri`. Apto para direcciones en línea, facilitando al usuario un teclado con la barra (/) y .com accesibles.
- `PostalAdress`. Especializado para direcciones físicas.
- `Text`. Mayúsculas en la primera palabra de la frase por defecto, multi-línea por defecto.

```
var contraseña by rememberSaveable { mutableStateOf("") }

OutlinedTextField(
    value = contrasena,
    onValueChange = { contraseña = it },
    label = { Text("Contraseña") },
    singleLine = true,
    keyboardOptions = KeyboardOptions(
        keyboardType = KeyboardType.PasswordVisible,
        imeAction = ImeAction.Done
    )
)
```

4.4. Selección y opciones

4.4.1. Checkbox

Permite al usuario seleccionar entre dos estados booleanos para cada opción presente. Puede utilizarse de forma aislada, o en un conjunto de opciones no-excluyentes.

```
var aceptado by remember { mutableStateOf(false) }

Row(verticalAlignment = Alignment.CenterVertically) {
    Checkbox(
        checked = aceptado,
        onCheckedChange = { nuevoEstado -> aceptado = nuevoEstado }
    )
    Text("He leído los términos y condiciones")
}
```

4.4.2. RadioButton

Permite seleccionar una única opción dentro de un grupo excluyente. Mientras que con XML existe una etiqueta rígida que gestiona la exclusividad (`RadioGroup`), en Compose se gestiona simplemente actualizando una variable compartida.

```
val opciones = listOf("Tarjeta", "Efectivo", "Bizum")
var opcionSeleccionada by remember { mutableStateOf(opciones[0]) }

Column {
    opciones.forEach { opcion ->
        Row(
            verticalAlignment = Alignment.CenterVertically,
            modifier = Modifier
                .fillMaxWidth()
                .clickable { opcionSeleccionada = opcion }
        ) {
            RadioButton(
                selected = (opcion == opcionSeleccionada),
                onClick = { opcionSeleccionada = opcion }
            )
            Text(text = opcion)
        }
    }
}
```

4.4.3. Switch

Elemento de activación/desactivación física inmediata, habitualmente empleado para configuraciones y preferencias.

```
var notificacionesActivas by remember { mutableStateOf(true) }

Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween,
    verticalAlignment = Alignment.CenterVertically
) {
    Text("Recibir notificaciones")
    Switch(
        checked = notificacionesActivas,
        onCheckedChange = { notificacionesActivas = it }
    )
}
```

4.4.4. Menús desplegables de selección

En el desarrollo clásico con XML, la selección dentro de un menú desplegable compacto se realizaba mediante el control `Spinner`. En Jetpack Compose, el estándar es `ExposedDropdownMenuBox`, que combina un campo de texto de solo lectura con un menú emergente.

```
val provincias = listOf("Madrid", "Barcelona", "Valencia", "Sevilla")
var expandido by remember { mutableStateOf(false) }
var provinciaSeleccionada by remember { mutableStateOf(provincias[0]) }

ExposedDropdownMenuBox(
    expanded = expandido,
    onExpandedChange = { expandido = !expandido }
) {
    OutlinedTextField(
        value = provinciaSeleccionada,
        onValueChange = {},
        readOnly = true,
        label = { Text("Provincia") },
        trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded =
expandido) },
        modifier = Modifier
            .menuAnchor()
            .fillMaxWidth()
    )

    ExposedDropdownMenu(
        expanded = expandido,
        onDismissRequest = { expandido = false }
    ) {
        provincias.forEach { provincia ->
            DropdownMenuItem(
                text = { Text(provincia) },
                onClick = {
                    provinciaSeleccionada = provincia
                    expandido = false
                }
            )
        }
    }
}
```

5. Colecciones dinámicas y listas perezosas

A menudo las aplicaciones Android requieren trabajar con colecciones de datos amplias y, generalmente, cambiantes o **dinámicos**. Mientras que los contenedores estáticos vistos anteriormente como Column y Row son perfectamente capaces de manejar datos comunes y preparados por el desarrollador, aquellas pantallas que manejen conjuntos dinámicos de datos requieren de tratamiento especializado.

5.1. Introducción al renderizado perezoso

Si colocamos una colección de 1000 elementos dentro de un `Column` estándar con desplazamiento vertical (`Modifier.verticalScroll(rememberScrollState())`), el compilador de Compose instanciará y compondrá los 1000 elementos en memoria de forma simultánea, incluso aunque en la pantalla del dispositivo solo quepan 5 a la vez; este funcionamiento acarrea consecuencias en la forma de un consumo desmedido de memoria RAM y la posibilidad de bloqueos y congelamiento de la aplicación.

Los **componentes perezosos** de Compose resuelven este problema mediante una estrategia de carga bajo demanda y reciclaje visual. Son los herederos del `RecyclerView` y los `Adapters` originales de XML.

5.2. Listas verticales

`LazyColumn` es el componente composable por excelencia para mostrar colecciones unidimensionales dispuestas verticalmente con el objetivo de hacer scroll para recorrer sus compoentes.

A diferencia de `Column`, dentro del cuerpo de este tipo de elementos no se pueden colocar composables sueltos directamente; se debe interactuar a través de su interfaz de construcción `LazyListScope`, la cual proporciona métodos específicos:

- `item { ... }`. Emite un único elemento individual (útil para cabeceras o pies de lista).
- `items(lista) { elemento -> ... }`. Itera sobre una colección de Kotlin tipo List, emitiendo un nodo visual por cada dato.
- `items(count = N) { index -> ... }`. Genera N elementos basados en un índice numérico.

```
data class Tarea(val id: Int, val titulo: String, val completada: Boolean)
@Composable
fun ListaTareasSimple() {
    // Colección dinámica en memoria
    val listaTareas = remember {
        listOf( Tarea(1, "Instalar Android Studio", true),
                Tarea(2, "Aprender Modifiers y Layouts", true),
                Tarea(3, "Dominar LazyColumn y LazyRow", false),
                Tarea(4, "Crear menú de navegación", false)
        )
    }
    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        // 1. Cabecera única
        item {
            Text(
                text = "Mis Tareas Pendientes",
                style = MaterialTheme.typography.headlineSmall,
                modifier = Modifier.padding(bottom = 8.dp)
            )
        }
        // 2. Iteración sobre la colección de datos
        items(listaTareas) { tarea ->
            TarjetaTarea(tarea = tarea)
        }
    }
}
@Composable
fun TarjetaTarea(tarea: Tarea) {
    Surface(
        modifier = Modifier.fillMaxWidth(),
        shape = RoundedCornerShape(8.dp),
        color = MaterialTheme.colorScheme.surfaceVariant
    ) {
        Row(
                modifier = Modifier.padding(16.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(
                    text = tarea.titulo,
                    style = MaterialTheme.typography.bodyLarge,
                    modifier = Modifier.weight(1f)
                )
                Checkbox(
                    checked = tarea.completada,
                    onCheckedChange = { /* Actualización de estado */ }
                )
            }
        }
    }
}
```

También existen `LazyRow`, que funciona exactamente con la misma API que `LazyColumn`, pero distribuye los elementos secuencialmente en el eje horizontal con desplazamiento lateral de izquierda a derecha.

5.3. Cuadrículas

Cuando los elementos no deben mostrarse en una única columna, sino organizados en una matriz bidimensional o cuadrícula, se utiliza `LazyVerticalGrid`. Es equivalente al `GridView` de XML.

Según el valor proporcionado al argumento `columns`, se establece el comportamiento de las casillas horziontales de la cuadrícula:

- `GridCells.Fixed(count)`. Fija un número constante de columnas exactas sin importar el ancho físico de la pantalla.
- `GridCells.Adaptive(minSize)`. Hace la interfaz adaptable, definiendo un ancho mínimo deseado para cada celda. El sistema calculará automáticamente cuántas columnas caben en pantalla según si el dispositivo es un móvil vertical, apaisado o una tableta.

```
@Composable
fun CuadriculaCategorias() {
    val categorias = remember {
        listOf("Hogar", "Electrónica", "Moda", "Deportes", "Libros")
    }

    LazyVerticalGrid(
        columns = GridCells.Fixed(2),
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalArrangement = Arrangement.spacedBy(8.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        items(categorias) { categoria ->
            Surface(
                modifier = Modifier
                    .fillMaxWidth()
                    .height(100.dp),
                shape = RoundedCornerShape(12.dp),
                color = MaterialTheme.colorScheme.primaryContainer
            ) {
                Box(
                    modifier = Modifier.fillMaxSize(),
                    contentAlignment = Alignment.Center
```

```
                ) {
                    Text(
                        text = categoria,
                        style = MaterialTheme.typography.titleMedium,
                      color = MaterialTheme.colorScheme.onPrimaryContainer
                    )
                }
            }
        }
    }
}
```

5.4. Optimización con claves únicas

Por defecto, cuando iteramos sobre una lista con `items(lista)`, Compose asocia internamente la posición sobre la que se está iterando al índice de cada elemento dentro de la lista.

Si el usuario interactúa con la colección en memoria realizando operaciones dinámicas como eliminar un elemento intermedio, insertar un nuevo elemento al inicio o reordenar la lista, **cambiarán inevitablemente los índices**.

Compose se verá obligado a recomponer o reconstruir los nodos de los elementos desplazados, ya que cree que el elemento que estaba en el índice 0 ha sido reemplazado por otro distinto. Esto no solo degrada el rendimiento, sino que puede provocar fallos visuales en animaciones o pérdida de estados temporales locales de las filas.

Para dotar de identidad permanente a cada fila, el método `items()` permite definir una clave única e inmutable vinculada a un identificador propio de la entidad.

```
LazyColumn {
    items(
        items = listaTareas,
        key = { tarea -> tarea.id }
    ) { tarea ->
        TarjetaTarea(tarea = tarea)
    }
}
```

6. Menús y barras de acciones

En el paradigma clásico basado en XML, las barras superiores y las acciones contextuales dependían de archivos de recursos dedicados en el directorio *res/menu/*, así como de la sobrescritura de métodos de *callback* en la actividad.

Sin embargo, para Jetpack Compose los menús y las barras de aplicaciones se integran como funciones @Composable dentro de la ranura correspondiente del componente `Scaffold` o anclados a componentes interactivos en pantalla.

6.1. Barra superior

La **barra superior** de la aplicación proporciona identidad visual, informa sobre la pantalla actual y concentra las acciones de navegación y comandos globales más utilizados.

El componente estándar para el desarrollo de la barra superior es `TopAppBar` (junto con sus variantes `CenterAlignedTopAppBar`, `MediumTopAppBar` y `LargeTopAppBar`). Se implementa habitualmente en la ranura *topBar* de un `Scaffold`.

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun BarraSuperiorEjemplo(
    titulo: String,
    onNavegarAtras: () -> Unit,
    onBuscarClick: () -> Unit,
    onAbrirMenuClick: () -> Unit
) {
    TopAppBar(
        title = {
            Text(text = titulo)
        },
        navigationIcon = {
            IconButton(onClick = onNavegarAtras) {
                Icon(
                    imageVector = Icons.AutoMirrored.Filled.ArrowBack,
                    contentDescription = "Volver a la pantalla anterior"
                )
            }
        },
```

```
        actions = {
            IconButton(onClick = onBuscarClick) {
                Icon(
                    imageVector = Icons.Default.Search,
                    contentDescription = "Buscar registros"
                )
            }
            IconButton(onClick = onAbrirMenuClick) {
                Icon(
                    imageVector = Icons.Default.MoreVert,
                    contentDescription = "Más opciones"
                )
            }
        },
        colors = TopAppBarDefaults.topAppBarColors(
            containerColor = MaterialTheme.colorScheme.primaryContainer,
          titleContentColor = MaterialTheme.colorScheme.onPrimaryContainer
        )
    )
}
```

Los componentes fundamentales de TopAppBar son los siguientes:

- *title*. Ranura destinada a un composable de texto que identifica el contexto de la pantalla activa.
- *navigationIcon*. Ranura situada al inicio (izquierda) reservada para la navegación de regreso en la pila de actividades.
- *actions*. Fila receptora de @Composable IconButton donde se declaran las acciones más relevantes. Si hay demasiadas opciones para el ancho disponible, la buena práctica consiste en reservar el último icono para desplegar un menú secundario.

6.2. Menús desplegables

El composable `DropdownMenu` representa una ventana emergente compacta que aparece superpuesta al contenido al pulsar un disparador. Se compone internamente de elementos `DropdownMenuItem`, y su visibilidad depende de una variable de estado booleana (expanded).

Tanto el selector de opciones del formulario visto en el punto 4.4.4. como este menú comparten el mismo motor interno: un contenedor que sirve de ancla, un desplegable flotante que responde al estado *expanded*, y una lista vertical de `DropdownMenuItem`.

```
@Composable
fun AccionesConMenuDesplegable() {
    // Variable observable que controla si el menú está visible u oculto
    var menuExpandido by remember { mutableStateOf(false) }

    Box {
        // Icono disparador (tres puntos)
        IconButton(onClick = { menuExpandido = true }) {
            Icon(
                imageVector = Icons.Default.MoreVert,
                contentDescription = "Opciones adicionales"
            )
        }
        // Menú emergente anclado al Box que contiene el botón
        DropdownMenu(
            expanded = menuExpandido,
            onDismissRequest = { menuExpandido = false } // Se cierra al
pulsar fuera o atrás
        ) {
            DropdownMenuItem(
                text = { Text("Configuración") },
                leadingIcon = {
                    Icon(Icons.Default.Settings, contentDescription = null)
                },
                onClick = {
                    menuExpandido = false
                    /* Ejecutar acción de configuración */
                }
            )
            DropdownMenuItem(
                text = { Text("Acerca de") },
                leadingIcon = {
                    Icon(Icons.Default.Info, contentDescription = null)
                },
               onClick = {
                   menuExpandido = false
                   /* Mostrar información */
               }
           )
           HorizontalDivider()
           DropdownMenuItem(
               text = { Text("Cerrar sesión") },
               onClick = {
                   menuExpandido = false
                   /* Finalizar sesión */
               }
           )
       }
   }
}
```

6.3. Menús contextuales

Un menú contextual no pertenece a la barra general de la aplicación, sino que está vinculado a un elemento concreto de la interfaz y permite realizar operaciones sobre él (editar, duplicar o eliminar).

En Jetpack Compose existen dos patrones habituales para disparar menús contextuales:

- **Menú contextual anclado mediante botón de acción en fila**. Cada elemento de una lista o tarjeta incluye un IconButton propio que despliega localmente su propio DropdownMenu.
- **Menú contextual por pulsación prolongada**. Equivalente directo al comportamiento del antiguo menú contextual de Android.